Project Mark-1

The mathematics of a learning, governed agent: what we are building, how it decides, and how it is trained. Draft for discussion, October 2026.

> **Read this first.** All graphs in this document come from a small simulation that we ran to illustrate the method, using fictional data (a laptop overheating example). They show how the mathematics behaves. They are not measurements of a product, and the real numbers will come from the demo and from real cases.

1. The idea on one page

A person points a phone at a device. Mark-1 recognises the exact unit, works out what is most likely wrong or what needs to be done, asks the most useful question or check next, and guides the person step by step. Every fact it uses comes from approved documents in an M-Files vault, always the version that applies to that exact unit. After each case, a person confirms the outcome and the system learns from it, so it needs fewer questions and makes better decisions over time.

Mathematically, Mark-1 is five pieces working together:

Piece	What it is, mathematically	What it does
Admissible knowledge	A filtered set of documents $K(u, t, h, g)$	Only approved, applicable, valid, permitted documents
Machine model	A directed graph $G = (V, E)$	Explains how failures spread (for example power)
Belief	A probability distribution $P(f \mid \text{observations})$ over causes	How sure it is about each possible cause
Decision rule	Maximise information gain per unit of cost	Chooses the next question or test
Learning	Count updates, practice in simulation, drift statistics, confidence bounds	Improves from confirmed cases and earns autonomy

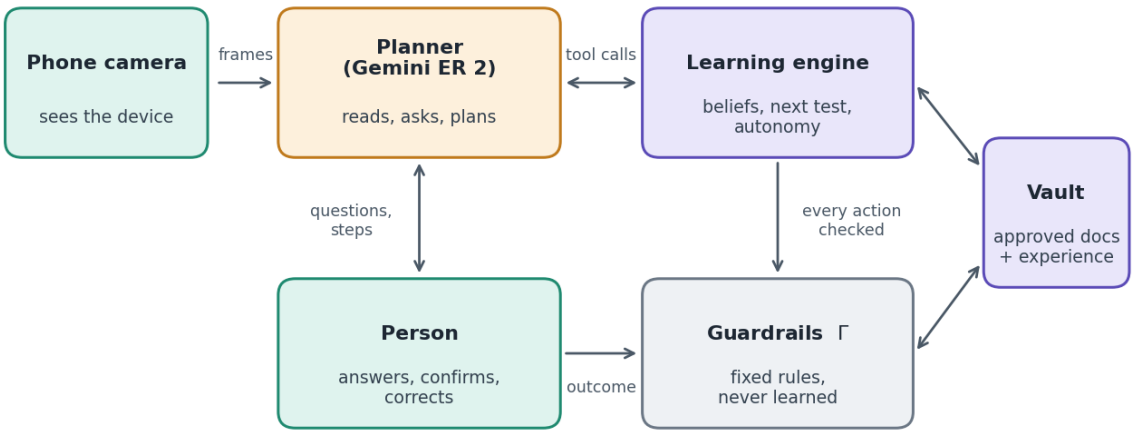


Figure 1. Architecture. The learning engine decides and learns. Every action it proposes passes the fixed guardrails Γ. The vault holds approved documents and the experience gathered from confirmed cases.

2. What we are building: the formal model

2.1 Units, documents and versions

Let u be a physical unit (a laptop with a given service tag), and $c(u)$ its configuration (model, board revision, software version). Let D be the set of documents in the vault. Every document object o has versions $o^{(1)}, o^{(2)}, \ldots$, and each version is in a workflow state s. Some states are released (for example Approved). The version that counts is the newest released one:

$$d^*(o) = o^{(v^*)}, \qquad v^* = \max\{\, v \,:\, s(o^{(v)}) \in S_{\text{released}} \,\}$$

Access in M-Files is built from several components, and all of them must grant access. For a person h and an agent identity g:

$$\text{acl}(d, x) = \bigwedge_{k=1}^{K} \text{grant}_k(d, x)$$

2.2 The admissible knowledge set

At time t, the only documents Mark-1 may use for unit u are:

$$K(u, t, h, g) = \{\, d \in D \,:\, \text{approved}(d) \wedge \text{applies}(d, c(u)) \wedge \text{valid}(d, t) \wedge \text{acl}(d, h) \wedge \text{acl}(d, g) \,\}$$

> **In plain words.** Before the agent thinks about anything, the vault narrows the world to documents that are approved, apply to this exact unit, are still valid today, and that both the person and the agent may see. Everything later in this document works only inside this set. This is where M-Files matters most: a smart agent reading the wrong revision is still wrong.

3. How the agent reasons

3.1 The machine as a graph

The digital blueprint is a directed graph $G = (V, E)$: nodes are components (charger, battery, board, CPU, fan), and an edge $i \to j$ means that j depends on i. Each component has a health variable $h_j \in \{0, 1\}$. Power propagates like this:

$$\text{pw}(j) = h_j \wedge \bigvee_{i \,:\, (i \to j) \in E} \text{pw}(i), \qquad \text{pw}(\text{source}) = 1$$

This lets the agent reason, not just match words. If the laptop shows a picture, the board has power, so a fault on the path before the board is ruled out. A fault hypothesis is kept only if what it predicts agrees with what the camera and the person report.

3.2 Belief over causes

Let F be the set of possible causes (for overheating: fan blocked by dust, fan motor failing, dried thermal paste, software load, sensor fault). Each check returns an observation o_i (for example "dust visible at the vents: yes"). Bayes' rule turns the observations into a belief:

$$P(f \mid o_{1:k}) \;\propto\; P(f) \prod_{i=1}^{k} P(o_i \mid f), \qquad \text{normalised so that } \sum_{f \in F} P(f \mid o_{1:k}) = 1$$

$P(f)$ is the prior: how common each cause is for this model, learned from past cases. $P(o \mid f)$ is the likelihood: how often a check comes out "yes" when the cause is f. This model treats checks

as independent given the cause, which is a simplification, discussed in section 7.

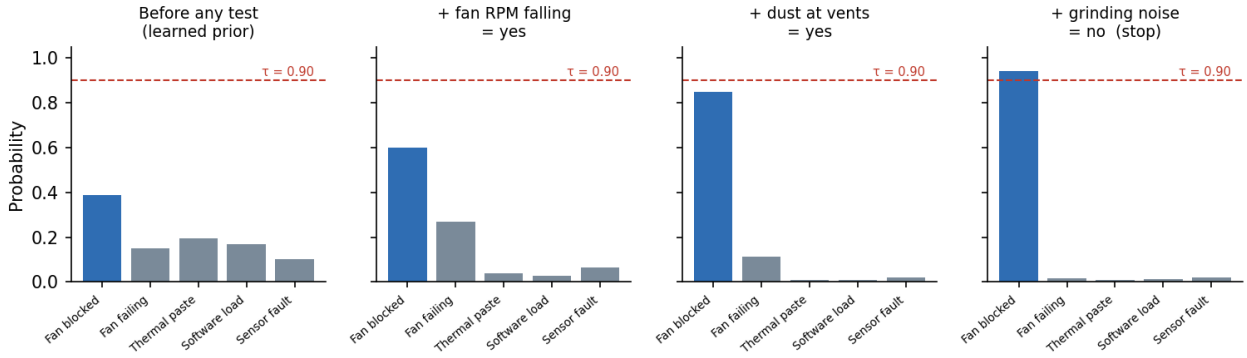


Figure 2. One simulated case. Each check moves the belief. After three checks, "fan blocked" reaches 94%, above the confidence threshold τ = 0.90, so the agent stops asking and proposes the approved cleaning procedure.

3.3 Choosing the next question or test

Uncertainty is measured with entropy, in bits. A belief spread evenly over five causes has about 2.3 bits of uncertainty; a belief certain of one cause has 0 bits:

$$H(F \mid o_{1:k}) = -\sum_{f \in F} P(f \mid o_{1:k}) \log_2 P(f \mid o_{1:k})$$

The value of a check *a* is how much uncertainty it is expected to remove, its information gain:

$$\mathrm{IG}(a) = H(F \mid o_{1:k}) - \sum_{y \in \{0,1\}} P(o_a = y \mid o_{1:k})\, H(F \mid o_{1:k}, o_a = y)$$

Checks have different costs: reading telemetry is almost free, while using an IR thermometer takes time. The agent picks the check with the best gain per unit of cost, only among checks the guardrails allow, and stops when it is confident enough:

$$a^* = \underset{a \in \mathcal{T}_{\text{allowed}}}{\operatorname{argmax}} \frac{\mathrm{IG}(a)}{\mathrm{cost}(a)}, \qquad \text{stop when } \max_f P(f \mid o_{1:k}) \geq \tau$$

If the budget of checks runs out before the threshold τ is reached, the agent does not guess. It escalates to a person with its current belief and the evidence.

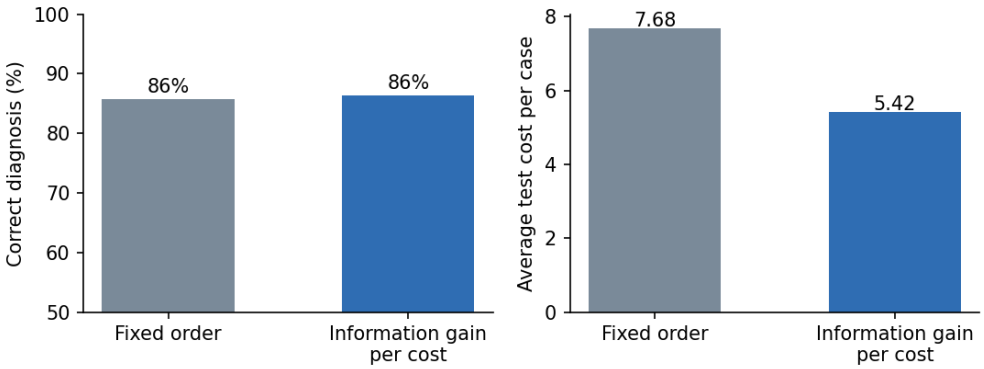


Figure 3. Simulation, 3,000 cases. Choosing checks by information gain per cost reaches the same accuracy (86% versus 86%) as following a fixed manual checklist, with about 29% less effort.

4. How the system is trained

Mark-1 learns in four ways. None of them changes the guardrails, and none of them requires retraining a large language model.

4.1 Starting knowledge from the documents

The vault's fault knowledge says, roughly, which checks are likely for which cause. We encode each document hint ℓ^{docs} as a starting count with strength m (how much we trust the documents at the start):

$$a_{t,f}^{(0)} = 1 + m\,\ell_{t,f}^{\text{docs}}, \qquad b_{t,f}^{(0)} = 1 + m\,(1 - \ell_{t,f}^{\text{docs}})$$

4.2 Learning from confirmed cases

After every repair, a person confirms the real cause $f^{\dagger}$. The engine keeps counts and turns them into probabilities:

$$\hat{P}(f) = \frac{n_f + \alpha_f}{N + \sum_{f'} \alpha_{f'}}, \qquad \hat{P}(o_t = 1 \mid f) = \frac{a_{t,f}}{a_{t,f} + b_{t,f}}$$

$$n_{f^\dagger} \leftarrow n_{f^\dagger} + 1, \qquad a_{t,f^\dagger} \leftarrow a_{t,f^\dagger} + o_t, \qquad b_{t,f^\dagger} \leftarrow b_{t,f^\dagger} + (1 - o_t)$$

In plain words. The system keeps a tally: how often each cause really happened, and how often each check said "yes" for each cause. Every confirmed case adds one to the tallies. The probabilities are just the tallies divided by the totals, with a small starting value from the documents, so they are never zero and never surprising at the start. Learning is instant, explainable ("based on 14 past cases"), and auditable, because each tally comes from a recorded case.

4.3 Practising in a training gym

Real faults are rare, so the engine also practises. From the blueprint and the fault knowledge we build a simulator P_{sim} that generates faults and the symptoms they would cause. The engine diagnoses each one, and is scored by a reward that values being right and penalises effort. Training searches for the best policy within the set Π_Γ of policies that obey the guardrails:

$$r = \mathbf{1}[\hat{f} = f^\dagger] - \lambda \sum_{a \in \text{tests}} \text{cost}(a), \qquad \pi^* = \operatorname*{argmax}_{\pi \in \Pi_\Gamma} \mathbb{E}_{f \sim P_{\text{sim}}}[\, r \,]$$

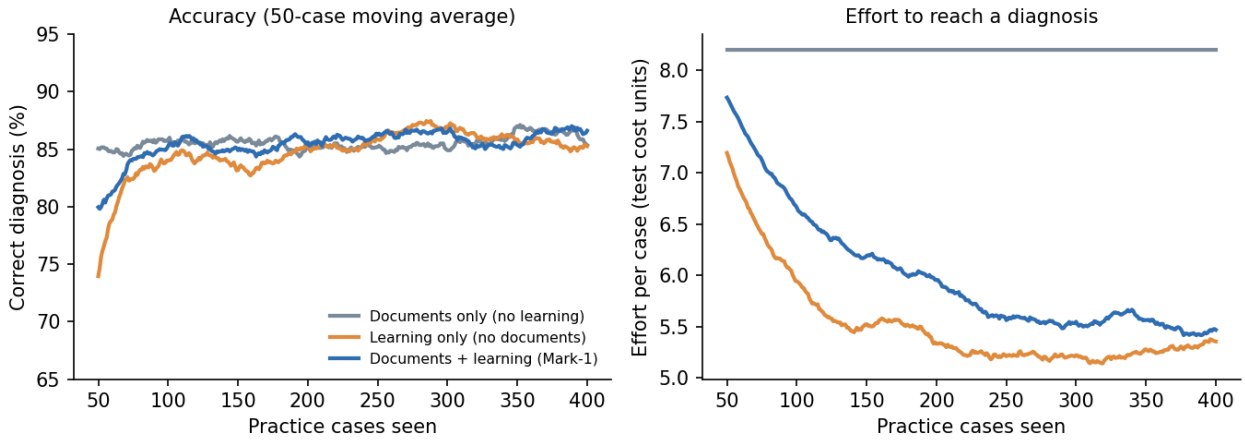


Figure 4. Training curves, simulation, averaged over 40 runs of 400 practice cases each. Left: accuracy. Right: effort per case. Grey: documents only, with no learning. Orange: learning only, starting from nothing. Blue: documents plus learning, which is the Mark-1 design.

What Figure 4 shows. With documents only, the agent reaches about 85% accuracy, but only by running every check on every case (effort 8.2): coarse document hints rarely make it confident enough to stop early. Learning only starts weak (74% on the first 50 cases), which would be unacceptable with real customers. Documents plus learning starts safe (80%), improves to 87%, and cuts effort from 7.7 to 5.5, about 29% less. The documents give a safe start; experience makes the agent efficient. The orange line ends slightly cheaper because the document prior keeps some caution; the strength *m* sets that trade-off between safety at the start and speed of adaptation.

4.4 Learning what normal looks like

For monitoring, the engine learns each unit's own baseline: the mean μ_0 and spread σ of a reading such as CPU temperature during normal weeks. It then accumulates small, persistent deviations with the CUSUM statistic:

$$S_t = \max(0,\ S_{t-1} + x_t - \mu_0 - k), \qquad \text{alarm if } S_t > h, \quad k = \tfrac{\sigma}{2},\ h = 5\sigma$$

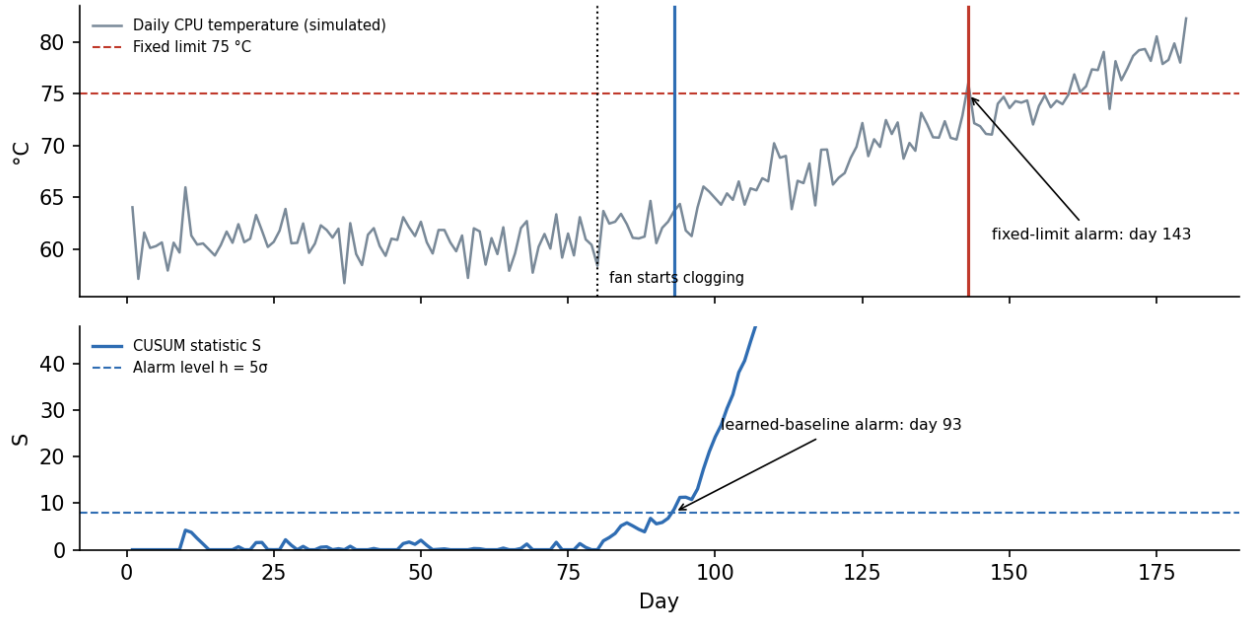


Figure 5. Simulation. A fan starts clogging on day 80. A fixed 75 °C limit raises an alarm on day 143. The learned baseline raises it on day 93, 50 days earlier, while the drift is still small.

In plain words. A fixed limit only reacts when things are already bad. A learned baseline knows what is normal for this particular laptop and notices a slow, steady change long before it crosses any fixed line.

4.5 Earning autonomy

The engine tracks its own record per task category: n decisions checked by a person, of which a fraction $\hat{p}$ were right. Because a short lucky streak should not count as proof, we use a conservative lower bound on the true accuracy, the Wilson score bound with $z = 1.96$ (95% confidence):

$$L = \frac{\hat{p} + \frac{z^2}{2n} - z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \qquad \text{level } \ell \text{ allowed if } L \geq \theta_\ell \ \wedge \ n \geq n_{\min} \ \wedge \ \text{approved}$$

Level	What the agent may do	Example promotion rule
0	Suggest only	Always allowed
1	Decide, then ask a person to confirm	L ≥ 0.80 and n ≥ 20
2	Decide and inform the person (low-risk tasks)	L ≥ 0.90 and n ≥ 30, approved by a person
3	Act alone in a proven low-risk category	L ≥ 0.97 and n ≥ 100, approved, re-checked regularly

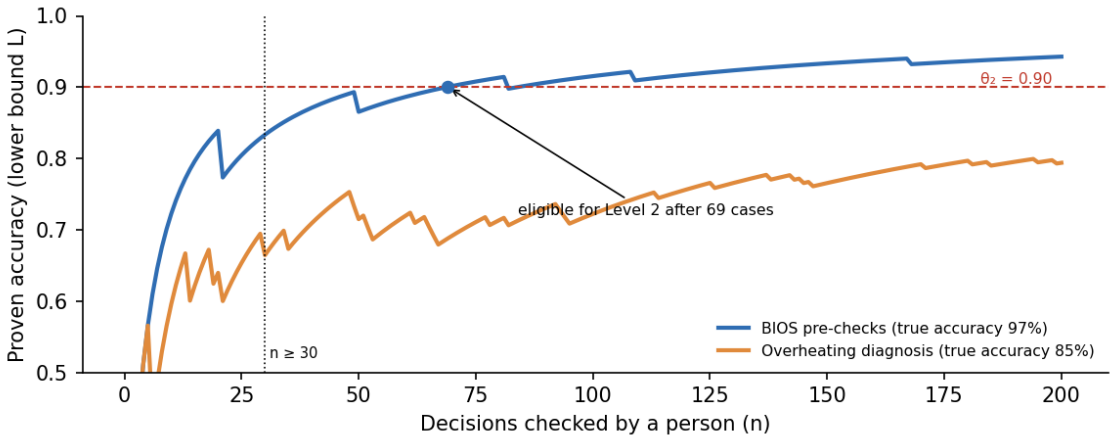


Figure 6. Simulation. BIOS pre-checks (true accuracy 97%) become eligible for Level 2 after 69 checked decisions. Overheating diagnosis (true accuracy 85%) never proves 90% and stays at Level 1. The thresholds above are proposals for discussion.

5. What never learns: the guardrails

Let Γ(*s*) be the set of actions allowed in situation *s*: use only admissible documents, respect hazard rules, never act while a required precondition fails, require a person for critical steps. The learned policy π_θ has parameters θ that change with experience, but the guardrails do not depend on θ:

$$\pi_\theta(a \mid s) = 0 \quad \text{for all } a \notin \Gamma(s), \qquad \frac{\partial \Gamma}{\partial \theta} = 0$$

> **In plain words.** Learning can change how the agent chooses among allowed actions. It can never add a forbidden action. This is how an experienced engineer works: judgment improves with every job, but the lockout procedure is never skipped. It is also what makes the system auditable: every rule is fixed and written down, and every learned number traces back to recorded cases.

6. How we will measure success

We define success with numbers before the demo, so the result can be checked:

Measure	Definition	Why it matters
Diagnostic accuracy	Share of cases where the top cause equals the confirmed cause	Is it right?
Effort per case	Sum of check costs before a decision	Is it efficient?
Calibration	Brier score, below, lower is better	When it says 90%, is it right about 90% of the time?
Right revision	Share of answers that used the correct document version	Target: 100%. A guardrail.
Correct refusal	Share of unclear or unknown cases that were escalated, not guessed	Safe behaviour
Early-warning lead time	Days between alarm and fixed-limit alarm; false alarms per unit-year	Value of monitoring
Autonomy record	Wilson bound L and n per task category	Evidence for promotion

$$\text{Brier} = \frac{1}{N} \sum_{j=1}^{N} \sum_{f \in F} \left(P_j(f) - \mathbf{1}[f = f_j^\dagger] \right)^2$$

7. What the simulation does and does not show

The simulation shows how the mathematics behaves when its assumptions hold. Four assumptions matter:

1. Checks are independent given the cause. In reality some checks are correlated, such as fan speed and noise. Grouping correlated checks or using a small Bayesian network fixes this.

2. The simulator knows the true fault model. Real faults are messier, so practice in the gym must be followed by validation on real, recorded cases.

3. Observations are binary and correct. Real readings from a camera can be wrong; we will measure how accurately Gemini reads labels, keys and gauges and include that error rate in the model.

4. Confirmed causes are correct. Learning depends on people confirming the real cause, so confirmation is a required step and is audited.

> **Bottom line.** The method is sound and standard: Bayes' rule, information gain, counting, CUSUM and confidence bounds have been used in engineering for decades. What is new is combining them with governed, versioned knowledge from M-Files and with an AI that can see. The numbers in the figures prove the method can work, not that it works on real devices yet.

8. From the mathematics to the 10-day demo

Demo example	Mathematics used	What the audience sees
BIOS update	Admissible set, preconditions in Γ, autonomy record	Policy checks, the right key blinks, refusal if a precondition fails
Overheating	Graph reasoning, Bayes, information gain, CUSUM, learning from cases	Early warning, "I ruled out X because Y", fewer questions after practice, x-ray view of the fan
Learning centrepiece	Training gym and learning curve, Wilson bound	Accuracy and effort improving live; a person approves a promotion to Level 2

Appendix A. Notation

Symbol	Meaning
u, c(u)	A physical unit and its configuration (model, board revision, software version)
D, K(u, t, h, g)	All vault documents; the admissible subset for unit u, time t, person h, agent g
$o^{(v)}$, $s(o^{(v)})$	Version v of a vault object, and its workflow state
G = (V, E), h_j	Blueprint graph of components and dependencies; health of component j
F, f, $f^{\dagger}$	Set of possible causes; one cause; the cause confirmed by a person
o_i, $P(f \mid o_{1:k})$	Observation i; belief over causes after k observations
H, IG(a), cost(a)	Entropy (uncertainty, bits); information gain of check a; its cost
τ	Confidence threshold to stop asking (0.90 in the simulation)
$n_{f'}$ $a_{t,f'}$ $b_{t,f'}$ m	Learned counts; strength of the document prior
π_θ, Γ(s), Π_Γ	Learned policy with parameters θ; allowed actions; policies obeying the guardrails
S_t, μ_0, σ, k, h	CUSUM statistic, learned baseline mean and spread, slack, alarm level
L, θ_ℓ, n_{min}	Wilson lower bound; threshold and minimum count for autonomy level ℓ

Appendix B. Simulation set-up

Domain: laptop overheating, five causes with frequencies 40%, 15%, 20%, 15%, 10% (fan blocked, fan failing, dried thermal paste, software load, sensor fault). Six binary checks with costs 0.2 (telemetry), 1 (look at vents), 1 (listen to fan), 1 (CPU load), 3 (IR thermometer) and 2 (load test); the most costly path through all checks is 8.2. Documents give coarse hints (0.7 likely, 0.3 unlikely) with two deliberate errors; document prior strength m = 4. Stop threshold τ = 0.90. Training curves: 40 independent runs of 400 cases, 50-case moving averages. Policy comparison: 3,000 cases after training; the fixed order follows a manual checklist (look, listen, load test, IR, CPU, telemetry). CUSUM: baseline learned from days 1 to 60, k = σ/2, h = 5σ, drift of 0.18 °C per day from day 80. Autonomy: 200 checked decisions per category. All data are fictional; the code is available in the project repository on request.

Project Mark-1. Draft for discussion. Not a product specification.